NGINX

Elaborada por: M. en C. Ukranio Coronilla

NGINX es un servidor web HTTP, servidor proxy, reverse proxy y balanceador de carga disponible en versión gratuita y también en versión empresarial conocida como **NGINX Plus** la cual incluye características avanzadas.

Antes de NGINX era común el uso del servidor **APACHE**, sin embargo, ocupaba muchos recursos al crear un proceso o un hilo por conexión, consumía mucha memoria y escalaba mal con miles de usuarios concurrentes.

Para la instalación de NGINX y verificación de esta ejecutamos:

```
sudo apt update
sudo apt install nginx
nginx -v
```

NGNIX como Servidor Web

En este caso NGNIX escucha peticiones HTTP, busca archivos en el disco, y devuelve HTML, CSS, JS e imágenes. A continuación, crearemos una página web sencilla para usar a NGINX como servidor Web.

Creamos la carpeta que contendrá los recursos html, css y js con:

```
sudo mkdir -p /var/www/ejemplo
```

Dentro de esta carpeta creamos los códigos html, css y js con el editor pico como superusuario. Por ejemplo, para el archivo `index.html` ejecutamos

```
sudo nano index.html
```

e introducimos el siguiente código:

index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
   <meta charset="UTF-8">
   <title>Nginx Servidor Web</title>
   <link rel="stylesheet" href="estilos.css">
</head>
<body>
   <h1>Hola desde Nginx</h1>
   <p>Este contenido es servido directamente por NGINX.</p>

   <button onclick="saludar()">Saludar</button>

   <script src="app.js"></script>
</body>
</html>
```

estilos.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #ffffff;
    padding: 40px;
}

h1 {
    color: #6a1b31;
}
```

app.js

```
function saludar() {
    alert("Nginx como servidor WEB.");
}
```

Al instalarse NGINX crea un archivo de configuración denominado default en la ruta:

```
/etc/nginx/sites-available/default
```

Dado que no lo utilizaremos y para evitar cualquier confusión pues crearemos nuestro propio archivo de configuración, procedemos a borrarlo con:

```
sudo rm /etc/nginx/sites-available/default
```

ahora lo creamos de nuevo en la carpeta sites-enabled (en realidad podría tener cualquier nombre, pero dejaremos por el momento el mismo nombre) e insertamos el siguiente código con:

`sudo pico /etc/nginx/sites-enabled/default`

default

```
server {
    listen 80;
    server_name localhost;

    root /var/www/ejemplo;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Con esta configuración le indicamos a NGINX que escuche en el puerto 80 (HTTP):

`listen 80;`

definimos el dominio local:

`server_name localhost;`

especificamos la carpeta raíz del sitio web:

`root /var/www/ejemplo;`

especificamos el archivo default cuando se acceda a http://localhost/ :

`index index.html;`

Por último, al servir archivos NGINX busca el nombre exacto del archivo, si no existe busca un directorio y si no existe devuelve `404 Not Found` con:

`try_files $uri $uri/ =404;`

Ahora para que nginx pruebe que el archivo de configuración es válido ejecutamos:

`sudo nginx -t`

Y para activar el servidor:

`sudo systemctl restart nginx`

Para acceder al sitio web de NGINX en el navegador ponemos:

http://localhost

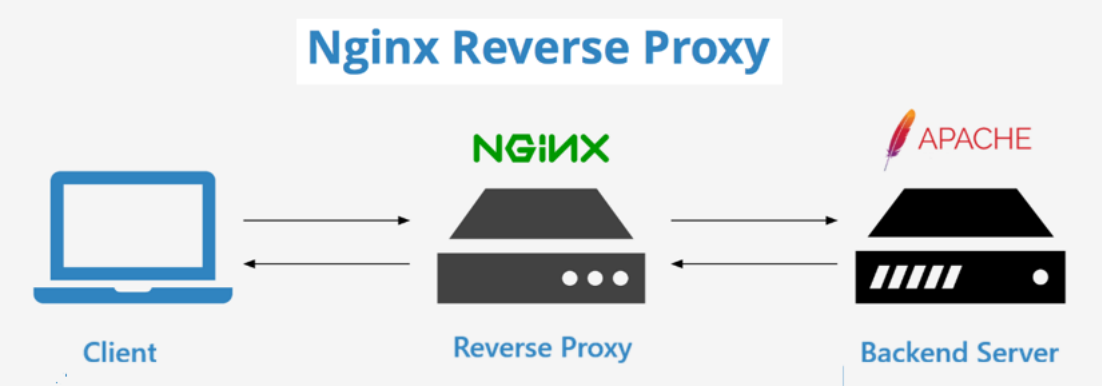
Para detener al servidor nginx ejecutamos:

`sudo nginx -s stop`

sin embargo, al reiniciar la terminal se volverá a ejecutar nginx a menos que eliminemos el archivo de configuración.

NGNIX como Reverse Proxy

Un reverse proxy (o proxy inverso) es un servidor que se sitúa entre los dispositivos de los usuarios y un servidor web (o varios de ellos) y es el uso más común de NGINX.



Entre las ventajas de usar NGINX como reverse proxy están las de que maneja miles de conexiones concurrentes sin ocupar muchos recursos pues no crea procesos ni hilos para cada conexión, y obtenemos mejor seguridad al no exponer las IP(s) del backend y sus puertos.

Para probarlo reescribimos el siguiente archivo de configuración:

default

```
server {
    listen 80;
    server_name localhost;

    location / {
        proxy_pass http://localhost:8080;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

En esta configuración además de escuchar en el puerto 80 y definir el dominio como localhost, le indicamos a NGINX con el código que encierran los corchetes después de

`location /`

lo que debe hacerse cuando la ruta inicia con /. La línea

`proxy_pass http://localhost:8080;`

indica que toda petición que llegue a NGINX se la envíe al servidor backend que estará disponible en `localhost:8080`. A continuación, NGINX agrega tres headers al mensaje HTTP que se envían al backend para que el backend sepa quién es el cliente externo. La primera línea envía el nombre del dominio original, la segunda la IP real del cliente externo y la tercera crea una cadena de proxies para propósitos de auditoría y microservicios.

Procedemos a activar el servidor NGINX ejecutando:

```
sudo nginx -t
```

```
sudo systemctl restart nginx
```

Si intentamos acceder con el navegador a localhost obtendremos el código **`HTTP 502 Bad Gateway`**, lo cual indica que un intermediario (Gateway o proxy) recibió una respuesta incorrecta del servidor final o no la recibió como era de esperarse.

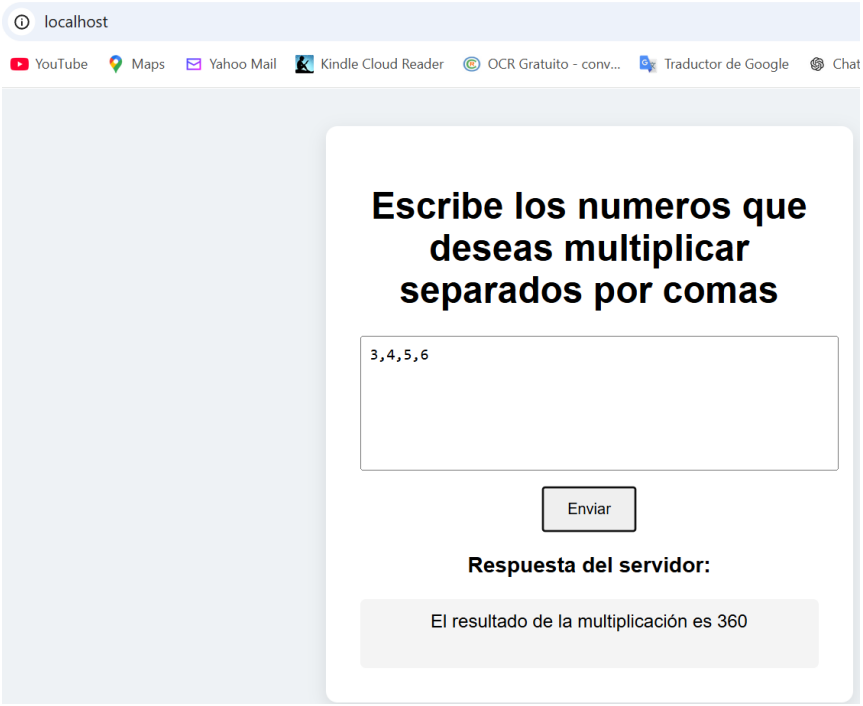
Ahora corremos nuestro servidor WebServer.java original el cual se ejecuta por default en el puerto 8080 como backend. Y en el navegador podremos acceder con

http://localhost/status

NGINX recibe la petición, la reenvía a nuestro servidor backend en Java el cual responde a NGINX y NGINX le devuelve la respuesta al cliente.

Ejercicio 1

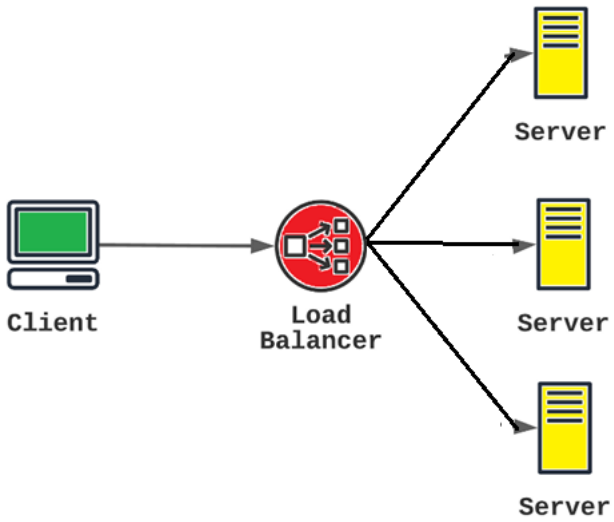
Con ayuda de la IA cree los códigos html, css y js además del archivo de configuración NGINX como reverse proxy para que sea posible acceder al endpoint `/task` del servidor desde un navegador que accede a http://localhost . Por ejemplo:



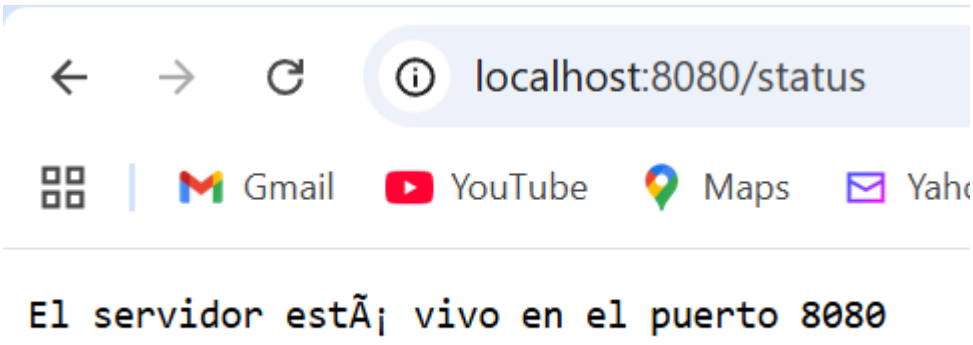
Envíe la captura de pantalla y los códigos html, css y js además del archivo de configuración NGINX.

NGINX como balanceador de carga

Un balanceador de cargas es un dispositivo (físico o software) que distribuye automáticamente el tráfico entrante proveniente de los clientes entre varios servidores o recursos backend para reducir la sobrecarga en un solo servidor, y garantizar una alta disponibilidad de servicios.



Para mostrar a NGINX como balanceador de carga modificaremos en WebServer.java la línea donde imprime `"El servidor está vivo\n";` y le agregaremos el puerto en el cual está escuchando de manera que al acceder al endpoint `/status` obtengamos:



Posteriormente corremos tres servidores en los puertos 8080, 8081 y 8082 y editamos el archivo de configuración para crear un balanceador de carga con:

```
sudo nano /etc/nginx/sites-available/loadbalancer
```

e introducimos el siguiente código:

```
# Grupo de servidores backend
upstream backend_java {
    server localhost:8080;
    server localhost:8081;
    server localhost:8082;
}

# Servidor frontal
server {
    listen 80;
    server_name localhost;

    location / {
        proxy_pass http://backend_java;
    }
}
```

Por una cuestión de orden NGINX utiliza dos carpetas:

`sites-available` donde se guardan todas las configuraciones de sitios que tengamos estén activos o no.

`sites-enabled` donde están las configuraciones que están activas y son leídas por NGINX. Para evitar conflictos con el ejercicio anterior eliminamos el archivo de configuración default de `sites-enabled` con:

```
sudo rm /etc/nginx/sites-enabled/default
```

y el archivo `loadbalancer` que está en `sites-available` en lugar de copiarlo a `sites-enabled` le creamos un enlace simbólico para no tener el archivo duplicado con:

```
sudo ln -s /etc/nginx/sites-available/loadbalancer /etc/nginx/sites-enabled/
```

Al listar los sitios activos con

```
ls -l /etc/nginx/sites-enabled/
```

Sólo deberíamos tener a loadbalancer aunque es un enlace simbólico. Con esta estrategia puedo tener varios archivos de configuración de NGINX almacenados en `sites-available` y seleccionar el que voy a utilizar al crear su enlace simbólico en `sites-enabled`.

Activamos la configuración:

```
sudo nginx -t
sudo systemctl restart nginx
```

Ahora probamos el balanceo ejecutando diez veces curl:

```
for i in {1..10}; do curl http://localhost/status; echo; done
```

Pruebe a ejecutar varias veces y compruebe que es del tipo Round Robin (por defecto) pero se puede balancear por IP del cliente donde el mismo cliente siempre accede al mismo servidor o donde a los servidores más potentes les asignamos más carga.

Ejercicio 2

Envíe la captura de pantalla donde se está probando a NGINX como balanceador de carga.